



Since Ray does everything already, what is something new which can be added or what is some gap? What is some novelty which can be built upon Ray alongside the other stuff that you've mentioned?

Good — I now have a rich picture. Let me give you the conceptual answer directly, then write the doc.

What Ray Doesn't Solve (And Where Compute4Me's Novelty Lives)

Ray is excellent, but it was built for one kind of world: a cluster you control, with nodes you trust, on hardware you provisioned, connected on a fast LAN or cloud network. The moment you step outside that world, Ray's assumptions start to crack. That's exactly where Compute4Me lives — and where there's genuine novelty to be built.

Here are the concrete gaps, each with what exists, what's missing, and what Compute4Me can contribute:

Gap 1: Open-World, Volunteer-Style Node Joining

What Ray does: Nodes join via `ray start --address=head_ip:port`. This requires the admin to provision each node, configure SSH, set up firewalls, and explicitly add it to the cluster. Ray has no concept of a node walking up and volunteering itself.^{[1] [2]}

What exists in the volunteer space:

- Hivemind (from the Learning@home project) trains large models on thousands of internet-connected volunteers using a peer-to-peer protocol (similar to BitTorrent's DHT for discovery). GitHub: <https://github.com/learning-at-home/hivemind>^{[3] [4]}
- BOINC and Folding@home handle volunteer compute for scientific simulations, but not DL training.^[5]
- Paper: "Distributed Deep Learning Using Volunteer Computing-Like Paradigm" (Atre et al., 2021): designed VC-ASGD, an async SGD algorithm suited to preemptible, unreliable volunteer-like nodes. arXiv: <https://arxiv.org/abs/2103.08894>^{[6] [7]}

The gap: Nobody has built a master-centric, Docker-native volunteer joining system specifically for DL. Ray is too ops-heavy. Hivemind is fully decentralized (no master). The

paper above uses preemptible cloud instances as a proxy for volunteers, not actual user-owned hardware joining via a container.

Compute4Me's novel contribution: A Docker image that any Ubuntu user can run, which automatically discovers the master, advertises its specs, and receives tasks — without requiring the master admin to touch anything on the volunteer's machine. This is genuinely novel infrastructure.

Gap 2: Capability-Aware, DL-Specific Heterogeneous Scheduling

What Ray does: Ray's scheduler is resource-based (you declare `num_cpus`, `num_gpus`, `memory` per task and Ray finds a node that has enough). But it treats all GPUs as equivalent — it doesn't know whether a node has 8 GB VRAM vs 24 GB VRAM, whether its PCIe bandwidth is fast, or whether its internet uplink is 10 Mbps or 1 Gbps. ^{[8] [9] [1]}

What exists:

- **Topology-aware GPU scheduling with deep reinforcement learning** (2025 paper): proposes a hybrid DRL + heuristic scheduler that achieves 47% improvement in throughput by considering GPU topology in its state representation. ScienceDirect: <https://www.sciencedirect.com/science/article/abs/pii/S0743731525001054> ^[10]
- **GPU Cluster Scheduling for Network-Sensitive Deep Learning** (2024): proximity-based scheduling that consolidates GPU resources by network distance to reduce communication bottlenecks. arXiv: <https://arxiv.org/html/2401.16492v1> ^[11]
- Netflix uses Ray for a heterogeneous training cluster but had to build custom layers on top. ^[8]

The gap: A scheduler that goes beyond "does it have a GPU?" to "what kind of GPU, how much VRAM, how fast is the network to this node, what's its historical task completion rate?" — and uses that to make intelligent decisions about *how much work* to assign (e.g., bigger data shards to the RTX 3070, smaller to the GTX 1060), and *what kind of parallelism* to use (data vs model vs trial). This is especially important in DL because VRAM is a hard constraint (model must fit in memory), not just a performance variable.

Compute4Me's novel contribution: A DL-aware capability profile per node (VRAM, TFLOPS, bandwidth, historical throughput) used by a custom scheduler to make heterogeneity-native placement decisions — something Ray's generic scheduler doesn't do.

Gap 3: SWARM-style Elastic Pipelines, but with a Master

What SWARM Parallelism does: SWARM is an algorithm for training large models on poorly connected, heterogeneous, unreliable devices. It splits the model into pipeline stages, but instead of fixed node-stage assignments, it **randomly re-wires** the pipeline when nodes fail or leave. arXiv: <https://arxiv.org/abs/2301.11913> ^{[12] [13] [14]}

Built on Hivemind, SWARM is fully decentralized. A 2025 follow-up from Pluralis Research showed that adding **asynchronous updates with NAG-based gradient correction** to SWARM

gives up to 45.9% improvement in wall-clock time and better stability on elastic swarms.^[15]

The gap: SWARM is masterless and requires models to be Hivemind-compatible (DMoE architecture). There is no system that combines:

- A logical master (for task decomposition, job submission UX, capability accounting)
- With SWARM-like elastic pipeline routing for model-parallel execution across heterogeneous nodes

Compute4Me's novel contribution: A hybrid architecture where a master decomposes and assigns jobs, but execution itself uses SWARM-style elastic pipelines among the worker containers — giving you both the UX/control of a master-centric system and the fault tolerance of SWARM.

Gap 4: Security and Trust for Semi-Trusted Volunteer Nodes

What Ray does: Ray assumes all nodes are trusted (same admin domain). There is no built-in mechanism to prevent a malicious worker from poisoning gradients or returning fabricated results.^{[16] [1]}

What exists:

- "Towards Volunteer Deep Learning: Security Challenges and Solutions" (2025): surveys exactly this problem — untrusted volunteer nodes in DL settings. HAL: <https://hal.science/hal-04879559v1/document>^[16]
- "Secure Distributed Training at Scale": proposes Byzantine-tolerant protocols with theoretical bounds on resistance to Byzantine and Sybil attacks.^{[7] [17]}
- Volunteer computing literature (survey, ACM 2019): discusses trust architectures for VC systems — centralized trust being easier to enforce but a bottleneck.^[18]

The gap: No system combines Byzantine-robust aggregation with a practical, container-based, volunteer DL fabric. Existing secure training papers are algorithmic; they don't come with a Docker-native implementation that works over commodity internet.

Compute4Me's novel contribution: Integrating lightweight Byzantine-robust aggregation (e.g., median-of-means or coordinate-wise median on gradients) directly into the master's aggregation step — so the master can statistically detect and discard bad gradient contributions from potentially unreliable or adversarial containers.

Gap 5: Cross-Paradigm Multiplexing on the Same Fabric

What Ray does: Ray Train handles training, Ray Tune handles HPO, Ray Serve handles inference. They all run on the same Ray cluster, but they're separate libraries with separate APIs. There's no unified "DL job" abstraction that automatically picks the right parallelism strategy based on the job type and current node availability.^{[19] [20]}

The gap: A single Compute4Me job submission could be:

- A training run → shard data across nodes, use DDP internally
- An HPO search → distribute trials, assign one trial per capable node
- A large model inference batch → pipeline-parallel inference

No current open-source system gives you a **single, simple job submission interface** that transparently picks and executes the right strategy based on the task description and real-time node capabilities.

Compute4Me's novel contribution: A job DSL or API where you describe *what* you want (train this model on this data, search these hyperparameters, run inference on this dataset) and the master figures out *how* to run it given the current volunteer pool — choosing data parallelism, model parallelism, trial parallelism, or a hybrid.

Summary Table: Ray vs Compute4Me's Novel Space

Dimension	What Ray Does	What Compute4Me Can Add
Node joining	Admin-provisioned, trusted	Docker-based volunteer self-registration
Scheduling	Generic resource matching (GPU count)	DL-aware: VRAM, bandwidth, history, TFLOPS
Elasticity	Supported but limited in heterogeneous settings	SWARM-style elastic pipelines + async NAG updates
Security	Trusted nodes only	Byzantine-robust aggregation for semi-trusted volunteers
Job interface	Separate Train/Tune/Serve APIs	Unified DL job DSL that auto-selects parallelism strategy
Hardware target	Managed clusters / cloud VMs	Commodity consumer GPUs (RTX 3070 etc.) over internet

Now writing this up as a doc:

✱

1. <https://www.usenix.org/system/files/osdi18-moritz.pdf>
2. https://www.alibabacloud.com/blog/best-practices-for-ray-clusters---ray-on-ack_600925
3. <https://learning-at-home.github.io>
4. https://www.reddit.com/r/StableDiffusion/comments/y2282l/learningathome_hivemind_decentralized_deep/
5. https://en.wikipedia.org/wiki/List_of_volunteer_computing_projects
6. <https://arxiv.org/abs/2103.08894>
7. <https://www.semanticscholar.org/paper/Distributed-Deep-Learning-Using-Volunteer-Paradigm-Atre-Jha/597926ab9dbd05baca191c409ca430047044297b>
8. <https://research.netflix.com/publication/heterogeneous-training-cluster-with-ray-at-netflix>
9. <https://www.anyscale.com/blog/four-reasons-why-leading-companies-are-betting-on-ray>

10. <https://www.sciencedirect.com/science/article/abs/pii/S0743731525001054>
11. <https://arxiv.org/html/2401.16492v1>
12. <https://arxiv.org/abs/2301.11913>
13. <https://proceedings.mlr.press/v202/ryabinin23a/ryabinin23a.pdf>
14. <https://openreview.net/forum?id=U1edbV4kNu>
15. <https://pluralis.ai/blog/swarm-parallel-async-updates/>
16. <https://hal.science/hal-04879559v1/document>
17. <https://ar5iv.labs.arxiv.org/html/2106.11257>
18. <https://dl.acm.org/doi/fullHtml/10.1145/3320073>
19. <https://github.com/ray-project/ray>
20. <https://www.anyscale.com/blog/distributed-deep-learning-with-ray-train-is-now-in-beta>
21. <https://www.youtube.com/watch?v=E98Pv36fOYI>
22. <https://arxiv.org/abs/1712.05889>
23. <https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0/vsphere-availability/providing-fault-tolerance-for-virtual-machines/fault-tolerance-requirements-limits-and-licensing.html>
24. <https://developer.nvidia.com/blog/efficiently-scale-llm-training-across-a-large-gpu-cluster-with-alpa-and-ray/>
25. <https://latitude.so/blog/ray-for-fault-tolerant-distributed-llm-fine-tuning>
26. https://escholarship.org/content/qt3r5069pj/qt3r5069pj_noSplash_e9af79b51aa5bfb303f19234f1e4c665.pdf
27. <https://www.youtube.com/watch?v=xxOZAAbftRM>
28. <https://dl.acm.org/doi/10.5555/3618408.3619631>
29. <https://openreview.net/forum?id=-azium0cV9>
30. <https://trepo.tuni.fi/handle/123456789/22225>
31. <https://forum.torproject.org/t/how-do-you-know-that-tor-nodes-are-not-run-by-the-same-entity/13156>
32. <https://www.semanticscholar.org/paper/5278b81db686b4d36143941bff1c683bea963a63>